

Architectural Dynamics of Dynamic Token Allocation Fallbacks and Cybernetic Control Loops in Autonomous Machine Intelligence

Theoretical Foundations of Cybernetic Control in Autonomous AI Systems

The deployment of modern large language models (LLMs) and distributed multi-agent swarms marks a transition in computing from deterministic execution to non-deterministic, adaptive dynamics. As autonomous agents are tasked with long-horizon execution—navigating unconstrained environments, orchestrating complex software tools, and performing multi-step reasoning—governing their operational stability, memory utilization, and computational spend demands principles derived from cybernetics and feedback control theory.

Requisite Variety and Context Window Engineering

W. Ross Ashby's Law of Requisite Variety establishes that for a regulator to achieve stability over a system, the internal variety (the volume of achievable states) of the regulator must equal or exceed the environmental variety of the disturbances it seeks to absorb. In generative AI architectures, the operational environment is defined by stochastic prompt inputs, dynamic API responses, and unpredictable tool execution outputs. The context window of an LLM functions as the primary variety bottleneck of the machine intelligence engine.

When environmental variety overpowers the model's internal processing capacity, systemic context rot occurs. This state is characterized by abrupt performance degradation once context utilization crosses structural thresholds, resulting in instruction drift, severe parameter hallucination, or premature task termination. System prompts, dynamic context retrieval, and formal JSON schemas act as structural variety attenuators and amplifiers. They reduce the unbounded state space of generative outputs into strictly executable representations while expanding the model's perceptual baseline to absorb environmental noise without collapsing internal operational stability.

Mapping the Viable System Model to Agent Governance

Stafford Beer's Viable System Model (VSM) provides an architectural blueprint for self-regulating, autonomous systems capable of maintaining structural viability within volatile environments. Mapping the five recursive subsystems of the VSM onto distributed multi-agent LLM infrastructures establishes a formal framework for system survival, resource coordination, and policy alignment across multi-agent workflows.

System 1 represents operational primary activities, encompassing individual LLM inference instances, low-level prefill and decode sequence generations, and domain-specific tool

executions. System 2 acts as the coordination mechanism, dampening operational oscillations and preventing resource contention between System 1 units. In modern inference topologies, System 2 functions through continuous micro-batch schedulers (such as vLLM or SGLang), rate limiters, and hardware memory management subroutines.

System 3 manages operational control, enforcing internal resource limits, allocating token budgets, and maintaining immediate execution stability. Embedded within System 3 is the System 3* audit channel, which periodically samples execution traces, validates output schemas, and monitors financial token expenditure without disrupting primary execution paths. System 4 is responsible for strategic intelligence and environmental adaptation. It scans the external environment to forecast system requirements, dynamically updating model routing strategies, restructuring schema parameters, and modifying context retrieval topologies based on broad task demands. System 5 defines systemic policy, values, and identity constraints. It arbitrates the structural tension between System 3's drive for operational efficiency and System 4's pressure for adaptive mutation. Without explicit System 5 governance, autonomous agents experience identity dissolution or drift into infinite tool-execution loops.

Algedonic Channels and Compute-Layer Governance

In cybernetic theory, an algedonic channel is an emergency feedback loop that bypasses standard administrative hierarchies to transmit immediate signals of severe pain (*algos*) or reward (*hedone*) directly from operational units (System 1) to policy setters (System 5). Standard operational telemetry propagates sequentially through System 2 and System 3, introducing aggregation delays that can prove fatal to execution safety. Algedonic channels act as immediate circuit breakers upon detecting critical operational or security threshold breaches. In enterprise agent architectures, algedonic governance is operationalized via non-bypassable sidecars embedded directly at the compute layer rather than the application layer. Placing enforcement at the compute layer ensures that controls cannot be bypassed regardless of prompt injection attacks or agent reasoning flaws. If an agent exhibits severe behavioral drift, attempts unauthorized data access, or enters execution loops that threaten resource exhaustion ("cloud bill shock"), the compute-layer algedonic signal immediately aborts processing. It revokes temporary execution credentials, collapses isolated compute environments, and escalates the system state to a mandatory Human-in-the-Loop (HITL) review gate. Conversely, algedonic pleasure signals (*hedone*) serve as synthetic reinforcement mechanisms. By caching successful execution traces and high-utility tool interaction paths within short-term memory ring buffers, the system dynamically reinforces optimal decision paths, preserving operational coherence over long reasoning horizons.

POSIWID and Closed-Loop Behavioral Realignment

Stafford Beer's foundational cybernetic dictum, "The Purpose of a System is What It Does" (POSIWID), emphasizes that a system's true operational purpose is defined by its observable runtime behavior rather than its designer's stated goals. For non-deterministic machine learning architectures, POSIWID implies that static prompt engineering and offline alignment are insufficient to guarantee safe runtime execution.

As models encounter complex dynamic environments, runtime behavior frequently diverges from developer intent. Closed-loop cybernetic governance relies on continuous output observation, quantitative error measurement, and dynamic input modulation to align actual

runtime behavior with nominal performance parameters.

Control-Theoretic Governance of Resource Allocation

Managing physical hardware resources during high-throughput LLM inference demands precise mathematical modeling. Generative inference is fundamentally bifurcated into a compute-bound prefill phase (processing prompt tokens to populate key-value caches) and a memory-bandwidth-bound decode phase (generating tokens autoregressively one by one). Unregulated serving environments suffer from queue saturation, severe memory bus contention, and tail-latency SLA breaches.

Mathematical Mechanics of Feedback Controllers in Serving Infrastructure

To regulate system variables such as GPU power caps, batch execution concurrency, and serving latency, classical feedback controllers are integrated directly into inference orchestrators. A standard Proportional-Integral-Derivative (PID) controller calculates an error signal $e(t)$ based on the difference between a target Service Level Agreement limit (SLA_{target}) and the observed performance metric ($PV(t)$):

$$e(t) = SLA_{\text{target}} - PV(t)$$

The continuous control output signal $u(t)$ is defined as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d e(t)}{dt}$$

where K_p , K_i , and K_d represent the proportional, integral, and derivative gain constants.

In discrete-time digital systems with a fixed sampling period T_s , the control law at time step k is calculated as:

$$u(k) = K_p e(k) + K_i \sum_{j=0}^k e(j) T_s + K_d \frac{e(k) - e(k-1)}{T_s}$$

Production deployment of feedback controllers requires continuous guardrails to ensure numerical stability and prevent actuator shock:

- Anti-Windup Compensation (Freeze Bias):** When physical control actuators hit hard execution boundaries $[u_{\text{min}}, u_{\text{max}}]$ —such as minimum GPU power limits or maximum concurrency caps—the control error can continue to accumulate within the integral term. This phenomenon, known as integrator windup, induces severe recovery delays and system oscillations once the error sign flips. Conditional anti-windup halts integral accumulation ($K_i \cdot e(k) = 0$) whenever the computed output saturates and the error signal shares the same algebraic sign as the control output.
- Error Deadbands:** High-frequency operational noise around setpoints can cause minor control outputs that induce unnecessary system jitter. Introducing an error deadband δ zero-out minor perturbations:
$$e(k) = \begin{cases} 0 & \text{if } |SLA_{\text{target}} - PV(k)| \leq \delta \\ SLA_{\text{target}} - PV(k) & \text{otherwise} \end{cases}$$
- Filtered Derivative Signals:** Raw telemetry metrics (such as instantaneous token generation latency) exhibit significant stochastic variation. Calculating derivatives on raw measurement noise induces severe output oscillations. Production implementations apply first-order low-pass filtering to the derivative term or calculate the derivative purely on the process variable rate of change (derivative-on-measurement) to prevent setpoint kick during operational mode transitions.

Edge System-on-Chip Bandwidth Regulation: The SERENO Framework

On mobile and edge System-on-Chip (SoC) platforms utilizing Unified Memory Architectures (UMA), the CPU, GPU, and NPU share a single DRAM memory pool. Executing background LLM inference concurrently with interactive foreground applications (such as user interface rendering or real-time gaming) introduces severe memory bandwidth contention.

Hardware firmware historically prioritizes NPU memory accesses, treating accelerator traffic as latency-critical. As a result, background LLM inference monopolizes DRAM access, degrading foreground user interface performance (surging frame jank rates by 153%) while background LLM throughput drops by less than 1.64%.

The SERENO architecture resolves this asymmetric resource interference by constructing a software-only control loop that uses speculative decoding draft phase latencies as an endogenous contention probe. The execution latency of speculative draft subgraphs correlates strongly ($R \approx 0.86$) with physical DRAM bandwidth contention, creating a near-zero-overhead sensor for systemic memory bus load.

SERENO deploys a Proportional-Integral (PI) control feedback loop to compute an aggregated yielding intensity $u(t)$:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau$$

where $e(t) = CS(t) - CS_{\text{target}}$, $CS(t)$ represents the real-time contention score derived from speculative draft probes, and CS_{target} is the maximum permissible contention threshold. The derivative (D) term is omitted in this application because stochastic mobile scheduling noise would cause the derivative calculation to amplify transient spikes, triggering erratic yielding decisions.

The controller output $u(t)$ is translated into phase-aware execution controls:

- **Draft Phase Actuation:** Micro-sleeps are disabled during speculative drafting to maximize token proposal generation. If $u(t)$ breaches a draft preemption threshold, the system halts speculative drafting immediately, releasing memory bus access to clear foreground UI rendering queues.
- **Verification Phase Actuation:** The control output $u(t)$ dynamically modulates micro-sleep durations inserted between model verification subgraphs, throttling NPU memory read frequencies to enforce smooth foreground frame delivery.

To ensure that background LLM task execution does not starve under heavy foreground workloads, SERENO incorporates a Token Bucket Minimum Throughput Guardrail that overrides PI control commands:

- **Nominal State:** When the token bucket contains tokens above a defined minimum reserve (e.g., 10% capacity), standard PI control rules enforce memory yielding.
- **Adaptive Relaxation State:** If the token bucket drops below its minimum reserve due to sustained yielding, the controller automatically relaxes CS_{target} , raising the contention threshold to restore background inference progress.
- **Fail-Safe State:** If a persistent token deficit occurs, the controller temporarily disables all memory yielding interventions, falling back to standard speculative decoding to prevent process starvation.

Micro-Architectural Scheduling & Dynamic Token

Budgeting

At the serving infrastructure layer, dynamic resource management bridges cybernetic feedback goals with kernel execution pipelines. High-performance LLM engines rely on specialized batch schedulers and dynamic token budgeting to maximize hardware efficiency under dynamic workload demands.

Engine Schedulers: vLLM and SGLang Mechanics

Modern serving frameworks implement dynamic context memory allocation and continuous batching schedulers to maximize parallel GPU execution:

- **Prefix Caching & RadixTree State Management:** SGLang uses RadixTree data structures to represent KV caches across prompt context hierarchies. When new requests arrive, the scheduler performs prefix matching over the RadixTree to reuse existing KV cache tensors, bypassing redundant prefill computations.
- **Chunked Prefill Orchestration:** Large prompt prefills can monopolize GPU compute units, causing active decode iterations to stall and spiking time-to-first-token (TTFT) metrics. Advanced schedulers resolve this by chunking long prompt prefills into discrete token allocations. A global token budget per iteration (e.g., 4096 tokens) is divided between processing prompt prefill chunks and executing single decode steps for active requests.
- **PrefillAdder Execution Logic:** In SGLang's execution pipeline, the PrefillAdder component continuously evaluates request memory footprints using the evaluation function: $\text{total_tokens} = \text{extend_input_len} + \min(\text{max_new_tokens}, 4096)$. The scheduler evaluates whether total_tokens can be allocated within available hardware KV cache blocks before launching execution kernels, preventing out-of-memory (OOM) process evictions.

Markov Decision Processes for Disaggregated Cluster Scaling

In enterprise deployments, inference instances are disaggregated into compute-bound prefill GPU pools and memory-bound decode GPU pools to maximize hardware efficiency.

Dynamically reallocating hardware nodes between prefill and decode pools is modeled as a Markov Decision Process (MDP) defined by the tuple $\langle S, A, P, R \rangle$,

where S is the state space, A is the action space, P is the transition probability, and R is the reward function.

The multi-dimensional state space $S_t \in S$ captures real-time operational load across four core system dimensions:

$S_t = \{ \lambda_t, \Delta \lambda_t, \tau_t, Q_{\text{len}}, C_{\text{none}} \}$

Traffic dynamics are tracked via current Requests Per Second (λ_t), traffic acceleration ($\Delta \lambda_t = \lambda_t - \lambda_{t-1}$), and cyclical temporal indicators (τ_t). System pressure is monitored via queue depth (Q_{len}) and allocation rejection frequency (C_{none}) caused by memory fragmentation.

A reinforcement learning agent monitors state S_t to execute dynamic scaling actions A_t , adjusting node allocations between prefill and decode pools. The reward function $R(S_t, A_t)$ penalizes latency SLA breaches and node migration overheads while maximizing aggregate request throughput.

Market-Based Allocation in Multi-Agent Swarms

When large multi-agent swarms compete for finite global token budgets, centralized scheduling introduces significant coordination overhead. Multi-agent architectures address this by deploying market-based double-auction mechanisms to distribute computational resources efficiently.

Participating agents receive periodic token budgets that serve as bidding currency. When submitting an inference request, an agent generates a bid B_i based on step criticality, remaining budget, and expected task reward R_i :

$$B_i = \frac{\text{Urgency}_i \cdot R_i}{\text{Remaining_Tokens}_i}$$

A decentralized clearing house executes continuous double-auctions at fixed time intervals, matching agent bids against available compute resources. High-bidding agents win access to top-tier, long-context models, while lower-bidding agents are dynamically routed to smaller localized models or queued for asynchronous batch processing.

Degradation Fallbacks and Context Management

As autonomous agents execute long-horizon multi-step tasks, accumulated interaction histories fill context windows, increasing inference costs and exposing models to context-limit pressure. Once context utilization crosses critical thresholds, model reasoning accuracy drops non-linearly. Preserving system continuity requires a multi-tiered context fallback hierarchy.

Context Rot Dynamics and Systemic Failure Modes

Empirical studies demonstrate that LLM reasoning capabilities degrade rapidly past structural context thresholds. Systemic failure modes manifest across three main behaviors:

- **Parameter Hallucination:** Models lose structural adherence to defined function schemas, inventing invalid parameters during tool invocation steps.
- **Instruction Drift:** High-priority system prompt constraints situated early in the context window lose attention weight relative to recent observation tokens.
- **Premature Task Termination:** Models sensing context saturation often declare task completion prematurely, returning partial outputs as successful final results.

Semantic Prompt Compression: LLMLingua Mechanics

The primary context fallback layer employs semantic prompt compression driven by lightweight auxiliary language models. Frameworks such as LLMLingua and LongLLMLingua compress context prompts prior to main LLM ingestion:

1. **Perplexity-Based Token Pruning:** A small calibrator model calculates conditional perplexity scores across input prompt sequences. Tokens exhibiting low surprisal (low conditional perplexity given surrounding context) are identified as syntactically redundant and pruned.
2. **Budget-Controlled Allocation:** Token reduction budgets are allocated dynamically across prompt components. Core instruction prompts retain high token densities, whereas historical tool outputs and observation logs undergo aggressive pruning. This dynamic allocation compresses prompt sizes by up to 80% while preserving underlying semantic intent.

Structural Schema Pruning and Context Filtering

Function call declarations and structured API schemas consume substantial portions of the context window in multi-tool agent environments. When schemas exceed allocation limits, dynamic structural pruning algorithms remove unused parameter declarations prior to model invocation.

Functional Dependency Graph Reranking: GRAST-SQL

In enterprise Text-to-SQL systems and complex API toolkits, injecting entire database catalogs introduces significant context noise and causes window exhaustion. The GRAST-SQL framework addresses this by constructing a Functional Dependency (FD) Graph over database schemas:

1. **Graph Construction:** Graph nodes represent database columns or tool parameters ($V = C$), while edges (E) capture foreign key constraints, primary key references, and predicted structural dependencies.
2. **Composite Relevance Scoring:** Given a natural language task query q , each schema table T_j receives a composite relevance score: $R(T_j, q) = \alpha \cos(e_q, e_T) + \beta PR(T_j) + \gamma Alias(T_j, q) - \eta ACLBlock(T_j, u)$ where $\cos(e_q, e_T)$ calculates vector semantic similarity between query intent and table metadata, $PR(T_j)$ represents PageRank structural centrality, $Alias(T_j, q)$ accounts for direct keyword matches, and $ACLBlock(T_j, u)$ penalizes tables restricted under user access control policies.
3. **Steiner Tree Subgraph Spanner:** Lexical and semantic retrieval methods frequently omit intermediate key columns required to execute valid database joins because primary/foreign key names rarely appear in natural language user queries. GRAST-SQL resolves this by executing a Steiner Tree spanner over the FD graph. The spanner isolates the minimal connected subgraph containing all high-relevance target nodes, preserving join paths while pruning irrelevant columns and tables.

Execution Verification Contracts

To prevent LLMs from hallucinating schema attributes or generating unexecutable code under pruned context constraints, agents enforce structural evidence contracts. Generated queries execute within isolated read-only sandboxes under strict time and resource limits. If execution fails or breaks schema constraints, execution error logs feed back directly into the generator model for a bounded number of self-correction attempts.

Integrated Systemic Blueprint

The structural interaction between control theory, cybernetic governance, micro-architectural scheduling, and context fallback mechanisms is synthesized across system operational tiers in the following architectural matrix:

Layer	System Variable & Operational Target	Sensor / Observation Probe	Primary Control Mechanism	Fallback & Circuit Breaker	Cybernetic Mapping (VSM)
Hardware Compute	DRAM Memory Bandwidth, UI Jank Rate	Speculative Drafting Latency Probes (R \approx 0.86)	SERENO PI Controller (Micro-sleep insertion & selective batching)	Token Bucket Throughput Guardrail (Adaptive threshold relaxation & fail-safe)	System 2 (Coordination)
GPU Power Capping	Tail Latency, SLA Violation Bounds	Real-Time Latency Distributions & Hardware Power Metrics	Adaptive PID Power Limit Modulation	Utilization-Aware Gating & Conditional Anti-Windup Freezing	System 3 (Operational Control)
Engine Scheduler	KV Cache Memory Blocks, Batch Concurrency	RadixTree Prefix Cache Telemetry, Scheduler Queue Depth (Q_{[span_246]}(start_span)[span_246](end_span)[span_247](start_span)[span_247](end_span)text{len}[span_157](start_span)[span_157](end_span)})	Continuous Batching & Chunked Prefills (PrefillAdder)	Dynamic Request Eviction & Allocation Rejection Logging (C_{\text{none}})	System 2 (Coordination)
Multi-Agent Swarm	Token Quotas, Shared Compute Budgets	Agent Marginal Utility Functions, Step Urgency	Continuous Double-Auction Clearing Markets	Dynamic Model Routing (Routing to local SLM sandboxes)	System 4 (Strategic Intelligence)
Context Memory	Context Window Saturation, Token Perplexity	Context Utilization %, Conditional Perplexity	Semantic Compression (LLMLingua perplexity pruning)	Steiner Tree FD Graph Reranking (GRAST-SQL schema filtering)	System 4 (Intelligence & Structure)
Compute Safety	Behavioral Alignment, Privilege Boundaries	Semantic Intent Vectors, API Call Telemetry	Non-Bypassable Compute-Layer Governance Sidecars	Algedonic Interrupt (Ephemeral cell teardown & HITL review	System 5 (Policy & Values)

Layer	System Variable & Operational Target	Sensor / Observation Probe	Primary Control Mechanism	Fallback & Circuit Breaker	Cybernetic Mapping (VSM)
				gate)	

Synthesis and Strategic Outlook

Governing autonomous machine intelligence requires moving beyond unconstrained open-loop generation toward closed-loop cybernetic governance. Implementing feedback control loops—incorporating proportional-integral regulation, anti-windup protection, and deadband guardrails—enforces predictable compute performance, hardware stability, and SLA adherence across fluctuating operational workloads.

Simultaneously, embedding Stafford Beer’s Viable System Model provides a robust framework for managing variety across agent software stacks. Separating real-time operational execution (System 1) and continuous batch coordination (System 2) from internal operational control (System 3), strategic context planning (System 4), and overarching value policy (System 5) yields resilient systems capable of long-horizon execution.

When combined with non-bypassable compute-layer algedonic sidecars and dynamic context fallback hierarchies, these systems maintain architectural stability, prevent execution runaway, and optimize resource allocation across modern enterprise environments.

Works cited

1. When control meets large language models: From words to dynamics, <https://arxiv.org/html/2602.03433v1> 2. The CASE Framework: A Multi-Disciplinary Control Architecture for, <https://arxiv.org/pdf/2608.10153> 3. AI Governance Platform Whitepaper | Algedonic.AI, <https://algedonic.ai/whitepaper> 4. vsm.md - GitHub Gist, <https://gist.github.com/tkellogg/3b9be96080526899363302c3bb87426f> 5. Towards Long-Horizon Agents: A Survey - Preprints.org, https://www.preprints.org/frontend/manuscript/1c2bdfc61b6dd77da822024ed82315ef/download_pub 6. The AI Agent Engineer's Guide: 60 Patterns for Building, <https://www.freecodecamp.org/news/ai-agent-engineers-guide-60-patterns-for-building-autonomous-systems-book/> 7. Viable system model - Wikipedia, https://en.wikipedia.org/wiki/Viable_system_model 8. Cybernetic AI Leadership with the Viable System Model, <https://www.wardleyleadershipstrategies.com/blog/ai-and-leadership/cybernetic-ai-leadership-with-the-viable-system-model> 9. Taming Memory Bandwidth Contention in Mobile LLM Inference with, <https://www.usenix.org/system/files/osdi26-xin.pdf> 10. vLLM V1, https://docs.vllm.ai/en/stable/usage/v1_guide/ 11. SGLang: Optimized LLM Execution & Caching - Emergent Mind, <https://www.emergentmind.com/topics/sglang> 12. (PDF) Self-Optimizing Enterprise RAG+SQL Knowledge Agents, https://www.researchgate.net/publication/413595888_Self-Optimizing_Enterprise_RAGSQL_Knowledge_Agents_Unified_Retrieval_Schema-Aware_Query_Translation_Event_Graph_Reasoning_and_Adaptive_Model_Routing 13. Pattern Recognition - Intelligent Organisations, <https://intelligente-organisationen.de/vsm-ai-pattern-recognition> 14. Energy-Efficient LLM Inference with SLA-Constrained Adaptive GPU, <https://www.computer.org/csdl/proceedings-article/compsac/2026/449700a140/2j6yG252wx2>

15. Agentic AI for Real-Time Adaptive PID Control of a Servo Motor - MDPI, <https://www.mdpi.com/2076-0825/14/9/459> 16. IJCAI-ECAI2026 PREPRINT - AWS, <https://ijcai-preprints.s3.us-west-1.amazonaws.com/2026/4269.pdf> 17. LLM Adaptive PID Control for B5G Truck Platooning Systems - MDPI, <https://www.mdpi.com/1424-8220/23/13/5899> 18. Dual-Mode Adaptive Defocusing Control for Net Energy Yield, <https://www.mdpi.com/1996-1073/19/15/3597> 19. PID Configuration Mode - AutomationDirect, <https://cdn.automationdirect.com/static/helpfiles/click/Content/243.htm> 20. Aircraft control with anti-windup compensation - SciSpace, <https://scispace.com/pdf/aircraft-control-with-anti-windup-compensation-feetfdjbre.pdf> 21. Industrial automation, robotics & controls Lingo - Umbrex, <https://umbrex.com/resources/industry-lingo/manufacturing-industrial-equipment-lingo/industrial-automation-robotics-controls-lingo/> 22. vLLM Explained: Why LLM Inference Is a Memory and Scheduling, <https://ylanglabs.com/blogs/vllm-inference-engine-explained> 23. [sglang-jax/docs/architecture/03-scheduler.md](https://github.com/sgl-project/sglang-jax/blob/main/docs/architecture/03-scheduler.md) at main - GitHub, <https://github.com/sgl-project/sglang-jax/blob/main/docs/architecture/03-scheduler.md> 24. MarketBench: Evaluating AI Agents as Market Participants - arXiv, <https://arxiv.org/html/2604.23897v1> 25. Market-Based Multirobot Coordination: A Survey and Analysis, https://www.researchgate.net/publication/2998069_Market-Based_Multirobot_Coordination_A_Survey_and_Analysis 26. Multi-Agent Reinforcement Learning for Decentralized ... - MDPI, <https://www.mdpi.com/2227-7390/14/11/1828> 27. Multi-Agent System Cost Control: Engineering Guide, <https://vector-labs.ai/insights/agent-cost-variability-is-a-engineering-problem-not-a-budget-line-how-to-build-swarms-that-dont-spiral> 28. Accelerating and Enhancing LLMs in Long Context Scenarios via, https://www.researchgate.net/publication/384208332_LongLLMLingua_Accelerating_and_Enhancing_LLMs_in_Long_Context_Scenarios_via_Prompt_Compression 29. A Deep Dive into Microsoft's LLMLingua Prompt Compression, <https://levelup.gitconnected.com/slash-your-llm-costs-by-80-a-deep-dive-into-microsofts-llmlingua-a-prompt-compression-d993c82009c4> 30. The 14th International Joint Conference on Natural Language, <https://aclanthology.org/events/aac1-2025/> 31. Prompt Compression for LLMs | FastRouter.ai, <https://fastrouter.ai/features/prompt-compression> 32. (PDF) Scaling Text2SQL via LLM-efficient Schema Filtering with, https://www.researchgate.net/publication/398850383_Scaling_Text2SQL_via_LLM-efficient_Schema_Filtering_with_Functional_Dependency_Graph_Rerankers